Linux Serial Ports Using C/C++

Unluckily, using serial ports in Linux is not the easiest thing in the world. When dealing with the `termios.h` header, **there are many finicky settings buried within multiple bytes worth of bitfields**. This page is an attempt to help explain these settings and show you how to configure a serial port in Linux correctly.

Everything Is A File

In typical UNIX style, **serial ports are represented by files** within the operating system. These files usually pop-up in `/dev/`, and begin with the name `tty*`.

Common names are:

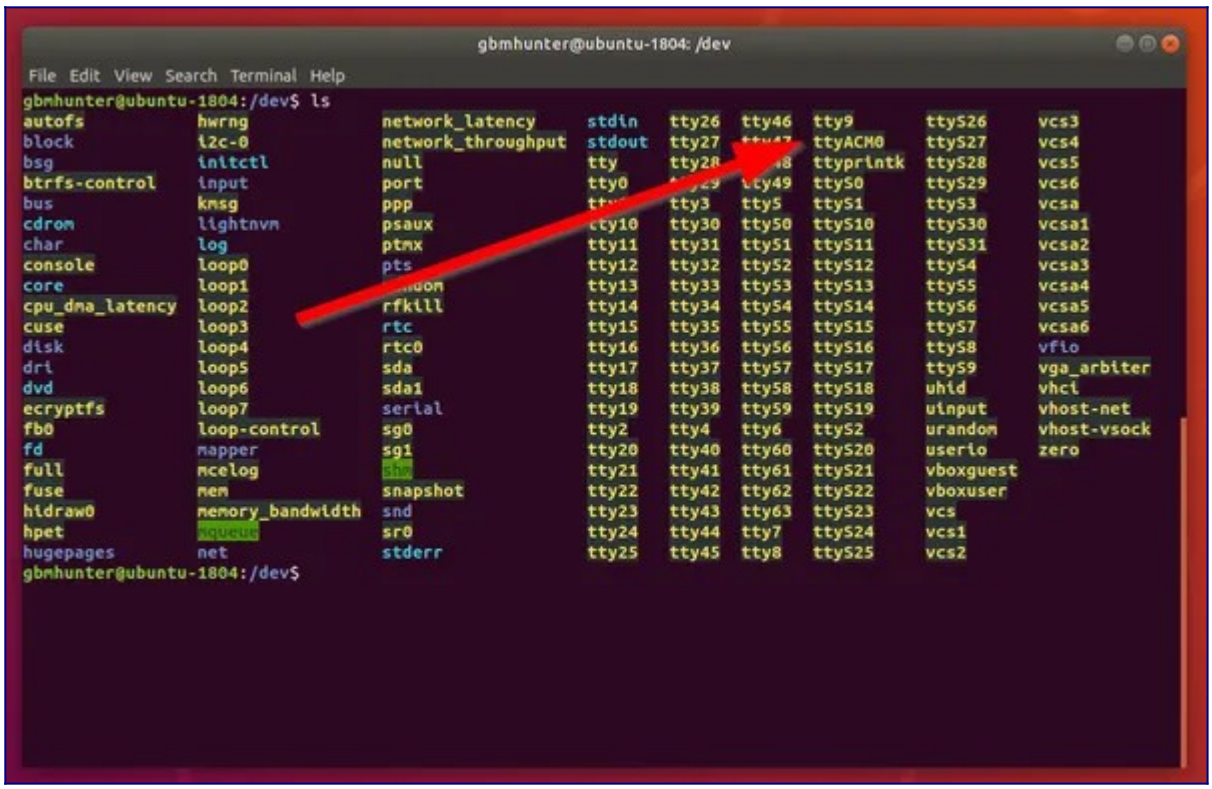
•`/dev/ttyACM0` - ACM stands for the ACM modem on the USB bus. Arduino UNOs (and similar) will appear using this name.

•`/dev/ttyPS0` - Xilinx Zynq FPGAs running a Yocto-based Linux build will use this name for the default serial port that Getty connects to.

•`/dev/ttyS0` - Standard COM ports will have this name. These are less common these days with newer desktops and laptops not having actual COM ports.

•`/dev/ttyUSB0` - Most USB-to-serial cables will show up using a file named like this.

•`/dev/pts/0` - A pseudo terminal. These can be generated with `socat`.



To write to a serial port, you write to the file. To read from a serial port, you read from the file. Of course, this allows you to send/receive data, but how do you set the serial port parameters such as baud rate, parity, etc.? This is set by a special `tty` configuration `struct`.

Basic Setup In C

Note

This code is also applicable to C++.

First we want to include a few things:

```
// C library headers
#include <stdio.h>
#include <string.h>

// Linux headers
#include <fcntl.h> // Contains file controls like O_RDWR
#include <errno.h> // Error integer and strerror() function
#include <termios.h> // Contains POSIX terminal control definitions
#include <unistd.h> // write(), read(), close()
```

Then we want to open the serial port device (which appears as a file under `/dev/`), saving the file descriptor that is returned by `open()`:

```
int serial_port = open("/dev/ttyUSB0", O_RDWR);

// Check for errors
if (serial_port < 0) {
    printf("Error %i from open: %s\n", errno, strerror(errno));
    return 1;
}
```

One of the common errors you might see here is `errno = 2`, and `strerror(errno)` returns `No such file or directory`. Make sure you have the right path to the device and that the device exists!

Another common error you might get here is `errno = 13`, which is `Permission denied`. This usually happens because the current user is not part of the dialout group. Add the current user to the dialout group with:

Terminal window

```
$ sudo adduser $USER dialout
```

You must log out and back in before these group changes come into effect.

At this point we could technically read and write to the serial port, but it will likely not work, because the default configuration settings are not designed for serial port use. So now we will set the configuration correctly.

When modifying any configuration value, it is best practice to only modify the bit you are interested in, and leave all other bits of the field untouched. This is why you will see below the use of &= or |=, and never = when setting bits.

Configuration Setup

We need access to the `termios` struct in order to configure the serial port. We will create a new `termios` struct, and then write the existing configuration of the serial port to it using `tcgetattr()`, before modifying the parameters as needed and saving the settings with `tcsetattr()`.

```
// Create new termios struct, we call it 'tty' for convention
// No need for "= {0}" at the end as we'll immediately write the existing
// config to this struct
struct termios tty;

// Read in existing settings, and handle any error
// NOTE: This is important! POSIX states that the struct passed to tcsetattr()
// must have been initialized with a call to tcgetattr() otherwise behaviour
// is undefined
if(tcgetattr(serial_port, &tty) != 0) {
    printf("Error %i from tcgetattr: %s\n", errno, strerror(errno));
}
```

We can now change `tty`'s settings as needed, as shown in the following sections. Before we get onto that, here is the definition of the `termios` struct if you're interested (pulled from `termbits.h`):

```
struct termios {
  tcflag_t c_iflag;    /* input mode flags */
  tcflag_t c_oflag;    /* output mode flags */
  tcflag_t c_cflag;    /* control mode flags */
  tcflag_t c_lflag;    /* local mode flags */
  cc_t c_line;      /* line discipline */
  cc_t c_cc[NCCS];    /* control characters */
};
```

Control Modes (c_cflags)

The `c_cflag` member of the `termios` struct contains control parameter fields.

PARENB (Parity)

If this bit is set, generation and detection of the parity bit is enabled. Most serial communications do not use a parity bit, so if you are unsure, clear this bit.

```
tty.c_cflag &= ~PARENB; // Clear parity bit, disabling parity (most common)
tty.c_cflag |= PARENB;  // Set parity bit, enabling parity
```

CSTOPB (Num. Stop Bits)

If this bit is set, two stop bits are used. If this is cleared, only one stop bit is used. Most serial communications only use one stop bit.

```
tty.c_cflag &= ~CSTOPB; // Clear stop field, only one stop bit used in
communication (most common)
tty.c_cflag |= CSTOPB;  // Set stop field, two stop bits used in communication
```

Number Of Bits Per Byte

The `CS<number>` fields set how many data bits are transmitted per byte across the serial port. The most common setting here is 8 (`CS8`) — definitely use this if you are unsure. I have never used a serial port that didn't use 8 (but they do exist). You must clear all of the size bits before setting any of them with `&= ~CSIZE`.

```
tty.c_cflag &= ~CSIZE; // Clear all the size bits, then use one of the
statements below
tty.c_cflag |= CS5; // 5 bits per byte
tty.c_cflag |= CS6; // 6 bits per byte
tty.c_cflag |= CS7; // 7 bits per byte
tty.c_cflag |= CS8; // 8 bits per byte (most common)
```

Hardware Flow Control (CRTSCTS)

If the `CRTSCTS` field is set, hardware RTS/CTS flow control is enabled. This is when there are two extra wires between the end points, used to signal when data is ready to be sent/received. The most common setting here is to disable it. Enabling this when it should be disabled can result in your serial port receiving no data, as the sender will buffer it indefinitely, waiting for you to be "ready".

```
tty.c_cflag &= ~CRTSCTS; // Disable RTS/CTS hardware flow control (most common)
tty.c_cflag |= CRTSCTS;  // Enable RTS/CTS hardware flow control
```

See the Software Flow Control section below on other settings relating to flow control.

CREAD and CLOCAL

Setting `CLOCAL` disables modem-specific signal lines such as carrier detect. It also prevents the controlling process from getting sent a `SIGHUP` signal when a modem disconnect is detected, which is usually a good thing here.
Setting `CREAD` allows us to read data (we definitely want that!).

```
tty.c_cflag |= CREAD | CLOCAL; // Turn on READ & ignore ctrl lines (CLOCAL = 1)
```

Local Modes (c_lflag)

Disabling Canonical Mode

UNIX systems provide two basic modes of input, **canonical** and **non-canonical mode**. In canonical mode, input is processed when a new line character is received. The receiving application receives that data line-by-line. This is usually undesirable when dealing with a serial port, and so we normally want to disable canonical mode.

Canonical mode is disabled with:

```
tty.c_lflag &= ~ICANON;
```

Also, in canonical mode, some characters such as backspace are treated specially, and are used to edit the current line of text (erase). Again, we don't want this feature if processing raw serial data, as it will cause particular bytes to go missing!

Echo

If this bit is set, sent characters will be echoed back. Because we disabled canonical mode, I don't think these bits actually do anything, but it doesn't harm to disable them just in case!

```
tty.c_lflag &= ~ECHO; // Disable echo
tty.c_lflag &= ~ECHOE; // Disable erasure
tty.c_lflag &= ~ECHONL; // Disable new-line echo
```

Disable Signal Chars

When the `ISIG` bit is set, `INTR`, `QUIT` and SUSP characters are interpreted. We don't want this with a serial port, so clear this bit:

```
tty.c_lflag &= ~ISIG; // Disable interpretation of INTR, QUIT and SUSP
```

Input Modes (c_iflag)

The `c_iflag` member of the `termios` struct contains low-level settings for input processing. The `c_iflag` member is an `int`.

Software Flow Control (IXOFF, IXON, IXANY)

Clearing `IXOFF`, `IXON` and `IXANY` disables software flow control, which we don't want:

```
tty.c_iflag &= ~(IXON | IXOFF | IXANY); // Turn off s/w flow ctrl
```

Disabling Special Handling Of Bytes On Receive

Clearing all of the following bits disables any special handling of the bytes as they are received by the serial port, before they are passed to the application. We just want the raw data thanks!

```
tty.c_iflag &= ~(IGNBRK|BRKINT|PARMRK|ISTRIP|INLCR|IGNCR|ICRNL); // Disable any special handling of received bytes
```

Output Modes (c_oflag)

The `c_oflag` member of the `termios` struct contains low-level settings for output processing. When configuring a serial port, we want to disable any special handling of output chars/bytes, so do the following:

```
tty.c_oflag &= ~OPOST; // Prevent special interpretation of output bytes (e.g. newline chars)
tty.c_oflag &= ~ONLCR; // Prevent conversion of newline to carriage return/line feed
// tty.c_oflag &= ~OXTABS; // Prevent conversion of tabs to spaces (NOT PRESENT IN LINUX)
// tty.c_oflag &= ~ONOEOT; // Prevent removal of C-d chars (0x004) in output (NOT PRESENT IN LINUX)
```

Both `OXTABS` and `ONOEOT` are not defined in Linux. Linux however does have the `XTABS` field which seems to be related. When compiling for Linux, I just exclude these two fields and the serial port still works fine.

VMIN and VTIME (c_cc)

`VMIN` and `VTIME` are a **source of confusion** for many programmers when trying to configure a serial port in Linux. They are designed to offer flexibility into how often the system call `read()` returns with received bytes, in the interests of reducing system call overhead and doing multi-byte reads. If `read()` returned one byte at a time this would have significant performance issues for fast data streams.

Both `VMIN` and `VTIME` can be 0 or a positive number. Let's explore the different combinations:

VMIN = 0, VTIME = 0: `read()` does not block, and will return immediately with what data is available. `read()` could return no bytes, or 1 or more bytes of data depending what was waiting in the buffer. This is a "polling" approach, and is useful if you want to check for data, but carry on doing something else if there is none. It is not recommended to repeatedly call `read()` in a loop in this mode, as this will burn through CPU cycles. If your application needs to do other things but also wait for serial data, you might want to look into one of the below blocking read approaches along with multiple threads.

VMIN > 0, VTIME = 0: This will make `read()` always wait for `VMIN` bytes before returning. `read()` could block indefinitely if not enough bytes are received.

VMIN = 0, VTIME > 0: This is a blocking read of any number of chars with a maximum timeout (given by `VTIME`). `read()` will block until either any amount of data is available, or the timeout occurs. If when the first byte is available, there are

also more bytes available in the input buffer, these will be returned at the same time as well. This happens to be my favourite mode (and the one I use the most).

VMIN > 0, VTIME > 0: In this mode, `read()` will block until either `VMIN` characters have been received, or `VTIME` has elapsed between characters. `VTIME` is called an "inter-character" timer in this mode. Note that the timeout for `VTIME` does not begin until the first character is received, and is reset every time a successive character is received[1].

`VMIN` and `VTIME` are both defined as the type `cc_t`, which I have always seen be an alias for unsigned char (1 byte). This puts an upper limit on the number of `VMIN` characters to be 255 and the maximum timeout of 25.5 seconds (255 deciseconds).

Note

"Returning as soon as any data is received" does not mean you will only get 1 byte at a time. Depending on the OS latency, serial port speed, hardware buffers and many other things you have no direct control over, **you may receive any number of bytes**.

For example, if we wanted to wait for up to 1s, returning as soon as any data was received, we could use:

```
tty.c_cc[VTIME] = 10;    // Wait for up to 1s (10 deciseconds), returning as
soon as any data is received.
tty.c_cc[VMIN] = 0;
```

It's also worth pointing out that `read()` will never return more than the number of bytes requested (`size_t count`, the third parameter passed into `read()`[2]). So in any of the above modes, as soon as it has `count` bytes, `read()` will return, even if `VMIN` is set to a higher number or the time specified by `VTIME` has not yet expired.

Baud Rate

Rather than use bit fields as with all the other settings, the serial port baud rate is set by calling the functions `cfsetispeed()` and `cfsetospeed()`, passing in a pointer to your `tty` struct and a `enum`:

```
// Set in/out baud rate to be 9600
cfsetispeed(&tty, B9600);
cfsetospeed(&tty, B9600);
```

If you want to remain POSIX compliant, the baud rate must be chosen from one of the following:

```
B0, B50, B75, B110, B134, B150, B200, B300, B600,
B1200, B1800, B2400, B4800, B9600, B19200, B38400
```

Linux and most BSD systems also provide higher-rate extensions outside POSIX (and a few even higher rates beyond these — `B500000`, `B921600`, `B1000000`, etc., on recent Linux kernels):

```
B57600, B115200, B230400, B460800
```

Some implementations of Linux provide a helper function `cfsetspeed()` which sets both the input and output speeds at the same time:

```
cfsetspeed(&tty, B9600);
```

Custom Baud Rates

As you are now fully aware that configuring a Linux serial port is no trivial matter, you're probably unfazed to learn that setting custom baud rates is just as difficult. There is **no portable way of doing this, so be prepared to experiment** with the following code examples to find out what works on your target system.

Integer-to-`cfsetispeed()` (BSD-style termios only — does NOT work on glibc Linux)

On some BSD-derived systems, `speed_t` is defined as the literal baud-rate integer (so `B9600` is `#define`d to `9600`), and `cfsetispeed()` / `cfsetospeed()` accept arbitrary integers directly:

```
// Works on BSD-style termios where speed_t == baud rate
cfsetispeed(&tty, 104560);
cfsetospeed(&tty, 104560);
```

This **does not work on modern glibc Linux**, where the `B...` macros are opaque speed codes (e.g. `B9600` is the value `13`, not `9600`). Passing a raw baud rate stores the wrong value in `c_ispeed` / `c_ospeed`, and the subsequent `tcsetattr()` will reject it with `EINVAL`. On Linux, use the `termios2` method below.

termios2 Method (the right way on Linux)

This method relies on using a `termios2` struct, which is like a `termios` struct but with slightly more functionality. I'm unsure on exactly what UNIX systems `termios2` is defined on, but if it is, it is usually defined in `termbits.h` (it was on the Xubuntu 18.04 with GCC system I was doing these tests on):

```
struct termios2 {
  tcflag_t c_iflag;    /* input mode flags */
  tcflag_t c_oflag;    /* output mode flags */
  tcflag_t c_cflag;    /* control mode flags */
  tcflag_t c_lflag;    /* local mode flags */
  cc_t c_line;      /* line discipline */
  cc_t c_cc[NCCS];    /* control characters */
  speed_t c_ispeed;    /* input speed */
  speed_t c_ospeed;    /* output speed */
};
```

This struct is very similar to plain old `termios`, except with the addition of `c_ispeed` and `c_ospeed`. We can use these to directly set a custom baud rate! We can pretty much set everything other than the baud rate in exactly the same

manner as we could for `termios`, except for the reading/writing of the terminal attributes to and from the file descriptor --- instead of using `tcgetattr()` and `tcsetattr()` we have to use `ioctl()`.

Let's first update our includes, we have to remove `termios.h` and add the following:

```
// #include <termios.h> This must be removed!
// Otherwise we'll get "redefinition of ‘struct termios’" errors
#include <sys/ioctl.h> // Used for TCGETS2/TCSETS2, which is required for custom baud rates
struct termios2 tty;

// Read in the terminal settings using ioctl instead
// of tcsetattr (tcsetattr only works with termios, not termios2)
ioctl(fd, TCGETS2, &tty);

// Set everything but baud rate as usual
// ...
// ...

// Set custom baud rate. The kernel-documented approach is:
//   1. Clear the speed bits in c_cflag (CBAUD)
//   2. Set BOTHER, telling the driver "use c_ispeed / c_ospeed for the rate"
//   3. Write the actual baud rate (in bits/s) to c_ispeed and c_ospeed.
tty.c_cflag &= ~CBAUD;
tty.c_cflag |= BOTHER;
tty.c_ispeed = 123456; // What a custom baud rate!
tty.c_ospeed = 123456;
// If BOTHER isn't defined on your system (very old kernels, some
// embedded toolchains), `tty.c_cflag |= CBAUDEX;` is sometimes used
// as a fallback — but BOTHER is the right answer on any modern Linux.

// Write terminal settings to file descriptor
ioctl(serial_port, TCSETS2, &tty);
```

Please read the comment above about `BOTHER`. Perhaps on your system this method will work!

Note

Not all hardware will support all baud rates, so it is best to stick with one of the standard `BXXX` rates above if you have the option to do so. If you have no idea what the baud rate is and you are trying to communicate with a 3rd party system, try `B9600`, then `B57600` and then `B115200` as they are the most common rates.

Saving termios

After changing these settings, we can save the `tty` termios struct with `tcsetattr()`:

```
// Save tty settings, also checking for error
if (tcsetattr(serial_port, TCSANOW, &tty) != 0) {
    printf("Error %i from tcsetattr: %s\n", errno, strerror(errno));
}
```

Reading And Writing

Now that we have opened and configured the serial port, we can read and write to it!

Writing

Writing to the Linux serial port is done through the `write()` function. We use the `serial_port` file descriptor which was returned from the call to `open()` above.

```
unsigned char msg[] = { 'H', 'e', 'l', 'l', 'o', '\r' };
write(serial_port, msg, sizeof(msg));
```

Reading

Reading is done through the `read()` function. You have to provide a buffer for Linux to write the data into.

```
// Allocate memory for read buffer, set size according to your needs
char read_buf [256];

// Read bytes. The behaviour of read() (e.g. does it block?,
// how long does it block for?) depends on the configuration
// settings above, specifically VMIN and VTIME
int n = read(serial_port, &read_buf, sizeof(read_buf));

// n is the number of bytes read. n may be 0 if no bytes were received, and can
also be negative to signal an error.
```

Closing

This is as simple as:

```
close(serial_port);
```

Full Example (Standard Baud Rates)

```
// C library headers
#include <stdio.h>
#include <string.h>

// Linux headers
#include <fcntl.h> // Contains file controls like O_RDWR
#include <errno.h> // Error integer and strerror() function
#include <termios.h> // Contains POSIX terminal control definitions
#include <unistd.h> // write(), read(), close()

int main() {
  // Open the serial port. Change device path as needed (currently set to a
standard FTDI USB-UART cable type device)
  int serial_port = open("/dev/ttyUSB0", O_RDWR);

  // Check for errors
  if (serial_port < 0) {
      printf("Error %i from open: %s\n", errno, strerror(errno));
      return 1;
  }
```

```
  // Create new termios struct, we call it 'tty' for convention
  struct termios tty;

  // Read in existing settings, and handle any error
  if (tcgetattr(serial_port, &tty) != 0) {
      printf("Error %i from tcgetattr: %s\n", errno, strerror(errno));
      close(serial_port);
      return 1;
  }

  tty.c_cflag &= ~PARENB; // Clear parity bit, disabling parity (most common)
  tty.c_cflag &= ~CSTOPB; // Clear stop field, only one stop bit used in
communication (most common)
  tty.c_cflag &= ~CSIZE; // Clear all bits that set the data size
  tty.c_cflag |= CS8; // 8 bits per byte (most common)
  tty.c_cflag &= ~CRTSCTS; // Disable RTS/CTS hardware flow control (most
common)
  tty.c_cflag |= CREAD | CLOCAL; // Turn on READ & ignore ctrl lines (CLOCAL =
1)

  tty.c_lflag &= ~ICANON;
  tty.c_lflag &= ~ECHO; // Disable echo
  tty.c_lflag &= ~ECHOE; // Disable erasure
  tty.c_lflag &= ~ECHONL; // Disable new-line echo
  tty.c_lflag &= ~ISIG; // Disable interpretation of INTR, QUIT and SUSP
  tty.c_iflag &= ~(IXON | IXOFF | IXANY); // Turn off s/w flow ctrl
  tty.c_iflag &= ~(IGNBRK|BRKINT|PARMRK|ISTRIP|INLCR|IGNCR|ICRNL); // Disable
any special handling of received bytes

  tty.c_oflag &= ~OPOST; // Prevent special interpretation of output bytes (e.g.
newline chars)
  tty.c_oflag &= ~ONLCR; // Prevent conversion of newline to carriage
return/line feed
  // tty.c_oflag &= ~OXTABS; // Prevent conversion of tabs to spaces (NOT
PRESENT ON LINUX)
  // tty.c_oflag &= ~ONOEOT; // Prevent removal of C-d chars (0x004) in output
(NOT PRESENT ON LINUX)

  tty.c_cc[VTIME] = 10;    // Wait for up to 1s (10 deciseconds), returning as
soon as any data is received.
  tty.c_cc[VMIN] = 0;

  // Set in/out baud rate to be 9600
  cfsetispeed(&tty, B9600);
  cfsetospeed(&tty, B9600);

  // Save tty settings, also checking for error
  if (tcsetattr(serial_port, TCSANOW, &tty) != 0) {
      printf("Error %i from tcsetattr: %s\n", errno, strerror(errno));
      close(serial_port);
      return 1;
  }

  // Write to serial port
  unsigned char msg[] = { 'H', 'e', 'l', 'l', 'o', '\r' };
  write(serial_port, msg, sizeof(msg));

  // Allocate memory for read buffer, set size according to your needs
  char read_buf [256];

  // Normally you wouldn't do this memset() call, but since we will just receive
  // ASCII data for this example, we'll set everything to 0 so we can
  // call printf() easily.
  memset(&read_buf, '\0', sizeof(read_buf));
```

```
  // Read bytes. The behaviour of read() (e.g. does it block?,
  // how long does it block for?) depends on the configuration
  // settings above, specifically VMIN and VTIME
  int num_bytes = read(serial_port, &read_buf, sizeof(read_buf));

  // num_bytes is the number of bytes read. It may be 0 if no bytes were
received, and can also be -1 to signal an error.
  if (num_bytes < 0) {
      printf("Error reading: %s", strerror(errno));
      close(serial_port);
      return 1;
  }

  // Here we assume we received ASCII data, but you might be sending raw bytes
(in that case, don't try and
  // print it to the screen like this!)
  printf("Read %i bytes. Received message: %s", num_bytes, read_buf);

  close(serial_port);
  return 0; // success
}
```

Issues With Getty

Getty can cause issues with serial communication if it is trying to manage the same `tty` device that you are attempting to perform serial communications with.

To Stop Getty:

Getty can be hard to stop, as by default if you try and kill the process, a new process will start up immediately.

These instructions apply to older versions of Linux, and/or embedded Linux.

1.Load `/etc/inittab` in your favourite text editor.

2.Comment out any lines involving `getty` and your `tty` device.

3.Save and close the file.

4.Run the command ~`$ init q` to reload the `/etc/inittab` file.

5.Kill any running `getty` processes attached to your `tty` device. They should now stay dead!

Exclusive Access

It can be prudent to try and prevent other processes from reading/writing to the serial port at the same time you are.

One way to accomplish this is with the `flock()` system call (note this example is in C++, easy to change to C if needed!):

```
#include <sys/file.h>

int main() {
```

```
    // ... get file descriptor here

    // Acquire non-blocking exclusive lock
    if(flock(fd, LOCK_EX | LOCK_NB) == -1) {
        throw std::runtime_error("Serial port with file descriptor " +
            std::to_string(fd) + " is already locked by another process.");
    }

    // ... read/write to serial port here
}
```

Getting The Number Of RX Bytes Available

You can use `FIONREAD` along with `ioctl()` to see if there are any bytes available in the OS input (receive) buffer for the serial port[3]. This can be useful in a polling-style method in where the application regularly checks for bytes before trying to read them.

```
#include <unistd.h>
#include <termios.h>

int main() {

  // ... get file descriptor here

  // See if there are bytes available to read
  int bytes;
  ioctl(fd, FIONREAD, &bytes);
}
```

The provided pointer to integer `bytes` gets written by the `ioctl()` function with the number of bytes available to be read from the serial port.

Changing Terminal Settings Are System Wide

Although getting and setting terminal settings are done with a file descriptor, **the settings apply to the terminal device itself and will affect all other system applications** that are using or going to use the terminal. This also means that terminal setting changes are persistent after the file descriptor is closed, and even after the application that changed the settings is terminated[4].

Examples

For Linux serial port code examples
see https://github.com/gbmhunter/CppLinuxSerial (note that this library is written in C++, not C).

External Resources

See GNU C Library – Terminal Modes for the official specifications of the `termios` struct configuration parameters.

1. The SCO Group Inc (2005). *Non-canonical mode input processing*. Retrieved 2023-12-16, from http://osr600doc.sco.com/en/SDK_sysprog/TDC_Non-CanonicalModeInProc.html. ↩
2. Michael Kerrisk (2023, Jun 24). *read(2) — Linux manual page*. Retrieved 2023-12-16, from https://man7.org/linux/man-pages/man2/read.2.html. ↩
3. Michael R. Sweet (1999). *Serial Programming Guide for POSIX Operating Systems*. Retrieved 2022-02-12, from https://www.cmrr.umn.edu/~strupp/serial.html. ↩
4. Free Software Foundation. *Mode Functions — The GNU C Library Reference Manual*. Retrieved 2023-12-16, from https://www.gnu.org/software/libc/manual/html_node/Mode-Functions.html